

Credit Name: CSE 3130 Object-Oriented Programming 2  
Assignment Name: bankAccount.java

**The following questions can help you in thinking critically about your problem-solving processes:**

## **Understanding the Problem**

### **How did you approach understanding the challenge?**

I approached understanding the challenge by first breaking the program into its main parts and identifying what each class was responsible for. I looked at the base account class to understand how balance, customer information, and basic methods like deposit and withdrawal worked. Then, I examined how the personalAccount and businessAccount classes extend the account class and override the withdrawal method to add their own rules for minimum balance fees. By tracing the flow in the main method step by step, I could see how objects were created and how methods were called, which helped me understand how inheritance and method overriding were being used to solve the problem.



I had to understand what I had to do first so I could apply it into the tester class when I made the code.

### **Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?**

I was initially confused about how the overridden withdrawal methods in the subclasses worked with the parent account method. I resolved this by tracing the steps of a withdrawal and using sample numbers to see how the balance changed after each operation, which helped me understand how the fee was applied.

```
// check if balance has fallen
if (getBalance() < 100) {
    // charge a $2 fee by with
    super.withdrawal(2.00);
}
```

To see if this worked I had to put in a sample number to see if it will actually take out more cash as a fee

## Planning the Solution

### Did you create a plan or break the problem into smaller steps before coding?

Yes, I broke the problem into smaller steps before coding. I first planned the base account class with balance and basic methods, then added the personalAccount and businessAccount classes with their specific rules. After that, I planned how the main method would create objects and test withdrawals, which made the overall program easier to build and catch any problems.

A screenshot showing two Java files in a code editor: personalAccount.java and businessAccount.java. The files are represented by small icons and text labels.

I started with the personal account so I can set up the variable and other things that are supposed to go to the main class. After I finished, I was able to repeat the same thing on the business account flawlessly.

### How did you decide on the tools, data structures, or algorithms to use?

decided on the tools and structure based on the requirements of the problem. Since the program involved different types of accounts with shared features, I used inheritance to create a base account class and specialized child classes. I used simple data types like double for balance and objects for customer information. For the logic, I used straightforward conditional statements to check minimum balances and apply fees, since the problem did not require complex algorithms.

```
-----  
private double balance;  
private customer cust;  
private String acctID;
```

## Implementation

### Did you write the code in small pieces or attempt the entire solution at once?

I wrote the code in small pieces rather than all at once. I started with the account class, then added the personalAccount and businessAccount classes, and finally worked on the main method. This step-by-step approach made it easier to test and fix errors as I went.

## How did you test your solution along the way to make sure it was working?

I tested my solution by running the program after completing each part and checking the output. I created sample accounts and performed withdrawals to see if the balances updated correctly and if the fees were applied when the minimum balance wasn't maintained. This helped me catch and fix errors early.

## Overcoming Challenges

### What part of the problem was the most difficult for you?

The most difficult part was correctly implementing the withdrawal logic with the minimum balance fees in the child classes. Making sure the fee was applied at the right time and that the balance updated correctly after both the withdrawal and the fee took some careful thinking and testing.

```
// withdraw 60 from personal account (may t
pAcc.withdrawal(60);

// display updated personal account informa
System.out.println(pAcc);

// display initial business account informa
System.out.println("\nBusiness Account:");
System.out.println(bAcc);

// withdraw 200 from business account (may
bAcc.withdrawal(200);

// display updated business account informa
System.out.println(bAcc);
```

Making sure the Main tester class knew to charge a fee took some time because when I was still testing the code it didn't charge a fee or do anything but I eventually fixed it because my variable wasn't set properly.

### How did you handle moments when you felt stuck or unsure of what to do next?

When I felt stuck, I broke the problem into smaller parts and focused on one piece at a time. I also reviewed my code and traced through it step by step to find where things might be going wrong. If needed, I tested small sections of the program to better understand the issue before continuing.

## **Learning**

**Was there anything you learned that you think will help you with future challenges?**

Yes, I learned how useful inheritance and method overriding are for organizing code and avoiding repetition. I also improved my ability to trace code step by step to understand how changes affect the program, which will help me debug and solve problems more efficiently in the future.